Carry-Sudoku & Slot-Sudoku in Reduced SHA-256: A Self-Contained ELI5 and ELI-Undergraduate Companion Guide

Companion Pedagogical & Reference Document to: *Practical Preimages for 19- and 20-Step Reduced SHA-256: Exact Algebraic Padding Inversion, Context Shaping, and the Carry/Slot-Sudoku Framework* (`PAPER_19_20_PADDED_SHA256.pdf`)

Executive Overview: Why a "Sudoku" Guide to SHA-256?

Cryptographic hash functions like SHA-256 are designed to work like one-way blenders: you drop a message in at the top, turn the crank for 64 rounds of bitwise mixing and 32-bit addition, and get a 256-bit fingerprint at the bottom. Trying to run the blender backward—starting from a target fingerprint and finding a validly padded input message that produces it (*a preimage attack*)—looks impossible by brute force, because there are 2^{256} possible target states.

However, when SHA-256 is reduced to **19 steps (padded)** or **20 steps (padded)**, you don't have to guess the whole message at once, and you don't have to fight the blender head-on. Instead, you can treat the internal step-by-register grid of SHA-256 like a **giant interconnected Sudoku puzzle**:

1. **You pre-fill strategic "givens" (fixed cells) in the middle of the board** (`e9 = e10 = e15 = 0xffffffff` , `a5 = a6` , `a10 = a11`) so that nonlinear logic gates (Ch and Maj) turn into simple pass-through wires or constants (`[E8]` – `[E13]` , `[E22]`).
2. **You solve the padding byte (`0x80`) in $O(1)$ time and solve the arithmetic carry chains bit-slice by bit-slice from right to left (`Carry-Sudoku`)** (`[E18]` – `[E26]`), where knowing the rightmost bits of e_{13} immediately forces the carries and eliminates 99.999% of invalid branches before the heavy rotations are even computed.
3. **You invert the coupled schedule operator $Q_0(u) = \sigma_0(u) - u$ across 3 chained steps using a zero-table 5-instruction bitwise inverse and a packed hash table (`Slot-Sudoku` / Cuckoo-Sudoku)** (`[E27]` – `[E37]`), eliminating memory bottlenecks on both Apple M3 CPUs and NVIDIA L4 GPUs.

This companion guide is organized so that **anyone**—from a curious reader to a cryptography undergraduate or systems engineer—can understand every single piece of the breakthrough:

- **Section 0 (Master Glossary & Complete `[E1]` – `[E41]` Index):** A 100% self-contained reference explaining every symbol, operator, C/CUDA variable, and numbered equation (`[E1]` through `[E41]`) in the main paper.
- **Section 1 — The Scalar a-Sequence Grid (`[E1]` – `[E7]`):** How the 8-register SHA-256 state collapses into a single 1D Sudoku strip $a_{-7}, \ldots, a_R$, and why FIPS 180-4 padding (`0x80` , zero fill, length `440`) locks the board (*ELI5* and *ELI-Undergraduate*).
- **Section 2 — Selector Saturation & Exact Context Shaping $D = 0$ (`[E8]` – `[E14]`):** How locking $e_9 = e_{10} =$ `0xffffffff` and forcing twin cells $a_5 = a_6$ ($D = a_5 \oplus a_6 = 0$) decouples the Left and Right halves of the cipher (*ELI5* and *ELI-Undergraduate*).
- **Section 3 — Exact $O(1)$ Algebraic Padding Inversion (`[E15]` – `[E22]`):** How to absorb $W_{15} = 440$ and $W_{14} = 0$ in closed form and solve $(W_{13} \bmod 256) =$ `0x80` in $O(1)$ time with zero trial-and-error (*ELI5* and *ELI-Undergraduate*).
- **Section 4 — `Carry-Sudoku` : 7-Stage Causal Bit-Slice & Rotation-Invariant Right-Context Solver (`[E23]` – `[E26]`):** How Theorem 1 splits $a_5 = a_6$ into a lower-triangular carry check independent of a_6, reaching **86.4 billion**

trials/s on Apple M3 and **56 trillion trials/s** on 1× NVIDIA L4 GPU (*ELI5* and *ELI-Undergraduate*).

- **Section 5 — `Slot-Sudoku` & Zero-Table 5-Instruction $\mathbb{F}_2$ Inverse (`[E27]` – `[E37]`):** How Theorem 2 reconstructs upper 16 bits in 5 register instructions (`reconstruct_u`), how 6 GB / 16 GB / 32 GB `Slot-Sudoku` tables work, and how every 19-step padded solution is filtered in $O(1)$ via σ_0^{-1} to find the first 20-step padded preimage (*ELI5* and *ELI-Undergraduate*).
- **Section 6 — GPU Warp-Sudoku & The Zero-a_3 Unpadded Bridge (`[E38]` – `[E41]`):** How 16-Way Single-T_{16} Fusion and Warp-Compacted Shared-Memory Queues reach **180,831.9 sol19/s** on 1× L4 GPU, and how setting $a_3 = 0$ solves 20-step unpadded compression preimages in 21.1 minutes on CPU (*ELI5* and *ELI-Undergraduate*).

Section 0: Master Self-Contained Glossary & Complete `[E1]` – `[E41]` Index

0.1 Core Symbols, Operators, and Code Variables

Symbol / Code Name	Name	Self-Contained Meaning in Plain English & Mathematics
$+, -$ (modulo 2^{32})	32-bit Modular Arithmetic	Addition and subtraction on 32-bit unsigned integers (`uint32_t` in C/CUDA), wrapping around modulo 2^{32}.
$\oplus, \wedge, \vee, \neg$	Bitwise Logic Operators	Bit-by-bit XOR (`^`), AND (`&`), OR (`|`), and bitwise NOT (`~`) across 32 bits ($b \in \{0, \dots, 31\}$).
$\mathrm{ROR}(x, n), \mathrm{SHR}(x, n)$	Right Rotate & Right Shift	Circular right rotation by n bits (`(x >> n) | (x << (32-n))`) and logical right shift (`x >> n`).
$\mathrm{Ch}(e, f, g)$	Bitwise "Choose" Gate	$(e \wedge f) \oplus (\neg e \wedge g)$. At each bit b, if $e_b = 1$, output f_b; if $e_b = 0$, output g_b. Setting $e =$ `0xffffffff` forces $\mathrm{Ch}(e, f, g) = f$, erasing g!
$\mathrm{Maj}(a, b, c)$	Bitwise "Majority" Gate	$(a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$. Outputs the bit (`0` or `1`) that appears at least twice. If two inputs are equal ($a = b$), $\mathrm{Maj}(a, a, c) = a$, erasing c!
$\Sigma_0(x), \Sigma_1(x)$	State Mixing Bijections	Invertible $\mathbb{F}_2$-linear state mixers: $\Sigma_0(x) = \mathrm{ROR}(x, 2) \oplus \mathrm{ROR}(x, 13) \oplus \mathrm{ROR}(x, 22)$ and $\Sigma_1(x) = \mathrm{ROR}(x, 6) \oplus \mathrm{ROR}(x, 11) \oplus \mathrm{ROR}(x, 25)$.
$\sigma_0(x), \sigma_1(x)$	Schedule Mixing Maps	$\mathbb{F}_2$-linear message mixers: $\sigma_0(x) = \mathrm{ROR}(x, 7) \oplus \mathrm{ROR}(x, 18) \oplus \mathrm{SHR}(x, 3)$ (invertible over $\mathbb{F}_2^{32}$!) and $\sigma_1(x) = \mathrm{ROR}(x, 17) \oplus \mathrm{ROR}(x, 19) \oplus \mathrm{SHR}(x, 10)$.
$Q_0(u), Q_1(u)$	Coupled Schedule Operators	$Q_0(u) := \sigma_0(u) - u \pmod{2^{32}}$ and $Q_1(u) := \sigma_1(u) + u \pmod{2^{32}}$. Used to invert the message schedule across steps 16, 17, 18.
$a_{-7}, \dots, a_R$	Scalar a-Sequence	Because 6 of the 8 SHA-256 registers simply shift each round, the entire 8-register trajectory is uniquely determined by the single sequence of a_s words (`[E5]` – `[E7]`).
$D_{\text{shape}} = a_5 \oplus a_6$	Context Shaping XOR	Measures how many bits differ between a_5 and a_6. Exact shaping sets $D_{\text{shape}} = 0 \iff a_5 = a_6$ (`[E11]`).
$M_3 = \neg(a_3 \oplus a_5)$	Level-3 Majority Mask	The bitmask of positions where a_3 and a_5 agree. On these bits, a_4 must equal a_5 so that $\mathrm{Maj}(a_5, a_4, a_3) = a_4$ (`[E13]`).
$D = a_8 \oplus a_7$	Carry-Sudoku Difference Mask	Inside Theorem 1 (`[E23]`), $D = a_8 \oplus a_7$ separates bits where $\mathrm{Maj}(a_8, a_7, a_6)$ depends on a_6 ($D_b = 1$) from bits where it equals $a_{8,b}$ ($D_b = 0$).

0.2 Complete Cross-Reference of Equations [E1] – [E41] in PAPER_19_20_PADDED_SHA256.pdf

- **[E1] (FIPS 180-4 Single-Block 55-Byte Padding):** Fixes $W_{15} = 440$ (0x000001b8), $W_{14} = 0$ (0x00000000), and $W_{13} \bmod 256 =$ 0x80 (72 locked padding bits).
- **[E2] (SHA-256 Boolean and Linear Functions):** Defines Ch, Maj, Σ_0, Σ_1, and σ_0, σ_1.
- **[E3] (Message Schedule Expansion):** $W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$ for $t \geq 16$.
- **[E4] (Coupled Schedule Operator Q_0):** Defines $Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}$.
- **[E5] (Initial Boundary Conditions):** Fixes $(a_0, a_{-1}, a_{-2}, a_{-3})$ and $(e_0, e_{-1}, e_{-2}, e_{-3})$ from IV.
- **[E6] (Terminal Boundary Conditions):** Fixes $a_{12} \ldots a_{19}$ (for $R = 19$) or $a_{13} \ldots a_{20}$ (for $R = 20$) from $H^* - \mathrm{IV}$.
- **[E7] (Scalar a-Sequence Recovery):** Expresses every e_s and W_t as a deterministic function of 5 consecutive a-words, leaving only $a_1 \ldots a_{12}$ free for $R = 20$ ($a_1 \ldots a_{11}$ for $R = 19$).
- **[E8] (Selector Saturation):** Pins $e_9 =$ 0xffffffff and $e_{10} =$ 0xffffffff to block e_7, e_8 from leaking rightward through Ch.
- **[E9] (Right-Only Expressions for a_5, a_6):** Expresses a_5 and a_6 purely in terms of the Right words (a_7, a_8, a_9, a_{10}).
- **[E10] (Collapsed Left Dependence of $W_9 \ldots W_{12}$):** Shows how $W_9, W_{10}, W_{11}, W_{12}$ depend on (a_1, a_2, a_3, a_4).
- **[E11] – [E12] (Exact Context Shaping $a_5 = a_6$):** Setting $D_{\text{shape}} = a_5 \oplus a_6 = 0$ ([E11]) forces $\mathrm{Maj}(a_6, a_5, a_4) = a_5$ ([E12]), making $W_{11} = c_{11} - a_3$ 100% free of a_4 leakage.
- **[E13] (Majority Cancellation in W_{12}):** Enforces $\mathrm{Maj}(a_5, a_4, a_3) = a_4 \iff ((a_4 \oplus a_5) \wedge M_3) = 0$ with $M_3 = \neg(a_3 \oplus a_5)$, making $W_{12} \ldots W_{19}$ 100% independent of (a_1, a_2, a_3, a_4).
- **[E14] (Three Chained Q_0-Inversions):** Rewrites the schedule equations at $t = 16, 17, 18$ as Level 1 ($Q_0(W_1) = T_{16} + F_1(a_1)$), Level 2 ($Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1$), and Level 3 ($Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2$).
- **[E15] – [E17] (Closed-Form Absorption of $W_{15} = 440, W_{14} = 0$):** Computes e_{16}, e_{15}, e_{14} ([E15]), solves e_{12}, e_{11} in closed form from e_{13} ([E16]), and recovers a_8, a_7 ([E17]).
- **[E18] – [E21] (Exact $O(1)$ Pivot Inversion of $W_{13} \bmod 256 =$ 0x80):** Uses the bit-disjointness of $\Sigma_1(e_{13}) \bmod 256$ on bit 18 and bits 25..31 ([E18] – [E20]) to pin those 8 bits in closed form ([E21]) for any choice of bits 0..17 and 19..24.
- **[E22] (Ch-Annihilation & Twin Majority Simplification):** Pins $e_{15} =$ 0xffffffff and $a_{10} = a_{11}$, making e_{12} and a_8 constant across all 2^{24} choices of e_{13}.
- **[E23] (Theorem 1 — Carry-Sudoku Exact Equivalence):** Proves $a_5 = a_6 \iff (((S \oplus X) \wedge D) = 0) \wedge (((a_6 \oplus S) \wedge \neg D) = 0)$, where $S = X + \mathrm{Ch}(D, X, a_8)$ is 100% independent of a_6 and $\Sigma_0(a_9)$.
- **[E24] – [E26] (7-Stage Causal Bit-Slice Pruning & Rotation-Invariant Windows):** Checks $((S \oplus X) \wedge D) = 0$ across 5 LSB-to-MSB bit-slices ([E24] – [E25]) and 2 rotation-invariant windows of $\Sigma_0(a_9)$ ([E26]).
- **[E27] – [E29] (Theorem 2 — Closed-Form 5-Instruction $\mathbb{F}_2$ Inverse reconstruct_u):** Solves the upper 16 bits u_{hi} from (y, u_{lo}) in 5 bitwise operations with zero lookup tables.
- **[E30] – [E33] (Slot-Sudoku Table Designs):** Defines the 6.00 GB 2-Tier Nested 4-Slot EDF + Cuckoo-Sudoku table ([E30] – [E31] , 82.98% coverage), the 16.00 GB Direct-Indexed 2-Way Exact GPU table ([E32] , 79.64% cov-

erage, zero false checks), and the 32.00 GB 4-Way Exact CPU table (`[E33]` , 99.57% coverage).

- **`[E34]` – `[E35]` (Early Hamming Pruning & 64-Bit SWAR Level-3 Filter):** Prunes $\text{popcount}(M_3) \leq w_{\text{thresh}}$ (`[E34]`) and tests 4 Level-3 slots simultaneously in one 64-bit register word (`[E35]`).
- **`[E36]` – `[E37]` (Terminal 20-Step Filter & σ_0^{-1} Bijection):** Filters the 19-step padded solution stream via $\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}$ (`[E36]`) and proves $\det_{\mathbb{F}_2}(\sigma_0) = 1 \implies W_4 = \sigma_0^{-1}(T_{19} - W_3)$ (`[E37]`).
- **`[E38]` – `[E41]` (Zero-a_3 Bi-Directional Q_0/Q_1 Bridge for Unpadded 20-Step Compression):** Pins $a_3 = a_5 = a_6 = a_8 = a_9 = 0, a_{10} = \texttt{0xffffffff}$ (`[E38]`) to annihilate both majority gates (`[E39]`) and solves the 20-step compression function (`[E40]` – `[E41]`) in 21.1 minutes on 16 CPU cores.

Section 1: The Scalar a-Sequence Grid (`[E1]` – `[E7]`)

1.1 Explain Like I'm 5 (ELI5): Turning an 8-Lane Highway into a Single Strip of Paper

If you look at a textbook diagram of SHA-256, it looks terrifying: there are **8 parallel lanes of numbers** (a, b, c, d, e, f, g, h) scrambling into each other for 20 steps, plus 16 message slots ($W_0 \dots W_{15}$) that expand into 4 recycled slots ($W_{16} \dots W_{19}$).

How do we turn that 8-lane mess into a clean Sudoku strip? Look closely at what happens to the 8 lanes in each step:

- Lane b is just a copy of lane a from 1 step ago! Lane c is lane a from 2 steps ago! Lane d is lane a from 3 steps ago!
- Similarly, lanes f, g, h are just copies of lane e from 1, 2, and 3 steps ago!
- Even better: lane e itself is completely determined by lane a (`[E7]`)!

That means **there aren't really 8 independent lanes at all**—there is only **one single strip of numbers**: $a_{-7}, a_{-6}, \dots, a_{20}$!

- The top 8 cells of the strip ($a_{-7} \dots a_0$) are pre-filled "givens" printed on the board by the SHA-256 starting value IV (`[E5]`).
- The bottom 8 cells of the strip ($a_{13} \dots a_{20}$ for 20 rounds, or $a_{12} \dots a_{19}$ for 19 rounds) are pre-filled "givens" locked by the target hash H^* (`[E6]`).
- **The entire 20-round SHA-256 puzzle comes down to filling in just 12 empty cells in the middle of the strip:** $a_1, a_2, \dots, a_{12}$ (or 11 cells $a_1 \dots a_{11}$ for 19 rounds) so that the padding rules (`[E1]`) and the 4 recycled message rules (`[E3]`) hold!

1.2 Explain Like I'm an Undergraduate (ELIU): Derivation of the Scalar a-Sequence (`[E5]` – `[E7]`)

In SHA-256, the step update at round $t \in \{0, \dots, R-1\}$ computes two new words:

$$T_{1,t} = h_t + \Sigma_1(e_t) + \text{Ch}(e_t, f_t, g_t) + K_t + W_t, \qquad T_{2,t} = \Sigma_0(a_t) + \text{Maj}(a_t, b_t, c_t),$$

and sets $a_{t+1} = T_{1,t} + T_{2,t}$ and $e_{t+1} = d_t + T_{1,t}$, while shifting $b_{t+1} = a_t, c_{t+1} = b_t, d_{t+1} = c_t$ and $f_{t+1} = e_t, g_{t+1} = f_t, h_{t+1} = g_t$. Subtracting $T_{1,t} = a_{t+1} - T_{2,t}$ from $e_{t+1} = d_t + T_{1,t}$ eliminates $T_{1,t}$ and W_t completely, giving the first equation of `[E7]` (with index $s = t+1$ and $d_t = a_{s-4}$):

$$e_s = a_s - \Sigma_0(a_{s-1}) - \mathrm{Maj}(a_{s-1}, a_{s-2}, a_{s-3}) + a_{s-4}.$$

Once e_s is known for all s, solving $e_{t+1} = d_t + h_t + \Sigma_1(e_t) + \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) + K_t + W_t$ for W_t gives the second equation of `[E7]` (where $d_t = a_{t-3}$ and $h_t = e_{t-3}$):

$$W_t = e_{t+1} - a_{t-3} - e_{t-3} - \Sigma_1(e_t) - \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) - K_t.$$

Therefore, once the boundary cells $a_{-7} \dots a_0$ (`[E5]`) and $a_{R-7} \dots a_R$ (`[E6]`) are fixed, **any** choice of the 12 interior words $(a_1, \dots, a_{12})$ defines a valid 20-step SHA-256 computation hashing to H^*. We only need $(a_1, \dots, a_{12})$ to satisfy:

1. The **72-bit FIPS 180-4 padding constraint (`[E1]`)**: $W_{15} = 440$, $W_{14} = 0$, and $W_{13} \bmod 256 = \texttt{0x80}$ (72 bit constraints).
2. The **4 message schedule recurrence equations (`[E3]`)** for $t = 16, 17, 18, 19$ ($4 \times 32 = 128$ bit constraints).

Section 2: Selector Saturation & Exact Context Shaping $D = 0$ (`[E8]` – `[E14]`)

2.1 Explain Like I'm 5 (ELI5): Building a Soundproof Wall in the Middle of the Board

We have 12 empty cells ($a_1 \dots a_{12}$) to fill in:

- Think of the first 4 cells (a_1, a_2, a_3, a_4) as **Team Left**.
- Think of the last 8 cells $(a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}, a_{12})$ as **Team Right**.

Why can't Team Right just solve the padding slots (W_{13}, W_{14}, W_{15}) first, and then let Team Left solve the recycled slots ($W_{16}, W_{17}, W_{18}, W_{19}$) afterward? Because normally, whenever Team Left changes (a_1, a_2, a_3, a_4), that change ripples rightward through e_5, e_6, e_7, e_8 and scrambles the middle slots $W_9, W_{10}, W_{11}, W_{12}$ and the padding slots W_{13}, W_{14}, W_{15}!

How do we build a **soundproof wall** between Team Left and Team Right?

1. **Lock the Railroad Switches (`[E8]`):** We set $e_9 = \texttt{0xffffffff}$ and $e_{10} = \texttt{0xffffffff}$. Remember how the Choose gate $\mathrm{Ch}(e, f, g)$ works: when e is all `1`s, it picks f and **disconnects the track from** g! Locking e_9 and e_{10} to all `1`s disconnects e_7 and e_8, stopping Team Left's ripples dead in their tracks!
2. **Make a_5 and a_6 Identical Twins (`[E11]` – `[E12]`):** Even with $e_9 = e_{10} = \texttt{0xffffffff}$, cell a_4 from Team Left tries to sneak into slot W_{11} through the Majority Vote gate $\mathrm{Maj}(a_6, a_5, a_4)$ (`[E10]`). How do we silence voter a_4? **We force voters a_5 and a_6 to cast the exact same ballot ($a_5 = a_6$, so $D_{\text{shape}} = a_5 \oplus a_6 = 0$)!** Since a_5 and a_6 agree on every bit, they win the 2-out-of-3 majority vote automatically ($\mathrm{Maj}(a_5, a_5, a_4) = a_5$), and a_4 is **100% locked out of W_{11}!**
3. **Cancel a_4 Out of W_{12} (`[E13]`):** In slot W_{12} (`[E10]`), a_4 appears twice with opposite signs: $+a_4 - \mathrm{Maj}(a_5, a_4, a_3)$. Whenever a_4 wins the vote $\mathrm{Maj}(a_5, a_4, a_3) = a_4$ (`[E13]`), $+a_4 - a_4 = 0$ cancels out completely!

2.2 Explain Like I'm an Undergraduate (ELIU): Formal Derivation of `[E8]` – `[E14]`

Let us trace equations `[E8]` through `[E14]` in detail:

1. **Selector Saturation (`[E8]` – `[E9]`):** Setting $e_9 = e_{10} = \texttt{0xffffffff}$ (`[E8]`) gives $\mathrm{Ch}(e_9, e_8, e_7) = e_8$, $\mathrm{Ch}(e_{10}, e_9, e_8) = \texttt{0xffffffff}$, and $\mathrm{Ch}(e_{11}, e_{10}, e_9) = \texttt{0xffffffff}$. Substituting $e_9 = e_{10} = \texttt{0xffffffff}$ into `[E7]` at $s = 9$ and $s = 10$ isolates a_5 and a_6 purely as functions of the Right variables (a_7, a_8, a_9, a_{10}) (`[E9]`):

$$a_5 = \texttt{0xffffffff} - a_9 + \Sigma_0(a_8) + \mathrm{Maj}(a_8, a_7, a_6),$$
$$a_6 = \texttt{0xffffffff} - a_{10} + \Sigma_0(a_9) + \mathrm{Maj}(a_9, a_8, a_7).$$

2. **Exact Context Shaping (`[E10]` – `[E13]`):** Expanding $W_9, W_{10}, W_{11}, W_{12}$ via `[E7]` under `[E8]` yields `[E10]` :

$$W_9 = c_9 - a_1, \qquad W_{10} = c_{10} - a_2,$$
$$W_{11} = c_{11} - a_3 - \big(\mathrm{Maj}(a_6, a_5, a_4) - (a_6 \wedge a_5)\big),$$
$$W_{12} = \mathrm{const}(\mathrm{Right}) + a_4 - \mathrm{Maj}(a_5, a_4, a_3).$$

- When we enforce **exact context shaping** $D_{\text{shape}} = a_5 \oplus a_6 = 0 \iff a_5 = a_6$ (`[E11]`), the identity $\mathrm{Maj}(a_5, a_5, a_4) = a_5 = a_6 \wedge a_5$ simplifies W_{11} to $W_{11} = c_{11} - a_3$ (`[E12]`) for **every** $a_4 \in \mathbb{Z}_{2^{32}}$.
- Simultaneously, in W_{12}, when does $\mathrm{Maj}(a_5, a_4, a_3) = a_4$? On any bit position where a_3 and a_5 are *opposite* ($a_{3,b} \neq a_{5,b}$), the majority of $(a_{5,b}, a_{4,b}, a_{3,b})$ is automatically $a_{4,b}$! On bit positions where $a_{3,b} = a_{5,b}$ (marked by the mask $M_3 := \neg(a_3 \oplus a_5)$), we simply require $a_{4,b} = a_{5,b}$, i.e., $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`). Then $+a_4 - \mathrm{Maj}(a_5, a_4, a_3) = 0$, so $W_{12}, W_{13}, \ldots, W_{19}$ are **100% independent of** (a_1, a_2, a_3, a_4)!

3. **The Three Chained Q_0-Inversions (`[E14]`):** Substituting $W_9 = c_9 - a_1$, $W_{10} = c_{10} - a_2$, $W_{11} = c_{11} - a_3$ and $W_t = a_{t+1} - G_t(a_1, \ldots, a_t)$ into `[E3]` for $t = 16, 17, 18$ produces the 3-level tree search (`[E14]`):

$$\text{Level 1 } (t = 16): \quad Q_0(W_1) = T_{16} + F_1(a_1),$$
$$\text{Level 2 } (t = 17): \quad Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1,$$
$$\text{Level 3 } (t = 18): \quad Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2.$$

Section 3: Exact $O(1)$ Algebraic Padding Inversion (`[E15]` – `[E22]`)

3.1 Explain Like I'm 5 (ELI5): Picking the 8 Secret Pins in the Padding Lock

Now Team Right $(a_5 \ldots a_{12})$ has to satisfy the postal inspector's three locked padding rules (`[E1]`):

1. $W_{15} = 440$ (message length),
2. $W_{14} = 0$ (zero block), and
3. The last byte of W_{13} must be the red STOP bead `0x80` ($W_{13} \bmod 256 = \texttt{0x80}$).

Here is how we unlock all three without guessing:

- First, once we pick a_{10}, a_{11}, a_{12} and a dial e_{13}, the equations for $W_{15} = 440$ and $W_{14} = 0$ (`[E16]`) immediately tell us the exact values of e_{12} and e_{11}!
- Second, what about the red STOP bead $W_{13} \bmod 256 = \texttt{0x80}$? Normally, random choices of e_{13} only hit `0x80` 1 time out of 256 ($1/256$).

- **The $O(1)$ Lockpick (`[E18]` – `[E21]`):** Look at how e_{13} is mixed inside W_{13}: it passes through $\Sigma_1(e_{13})$, which rotates e_{13} right by 6, 11, and 25 bits (`[E20]`). When we look at the lowest 8 bits of $\Sigma_1(e_{13})$, we discover a miracle: **bits 25..31 and bit 18 of e_{13} each touch exactly one bit of the low byte of W_{13}, and never touch the low byte of e_{13} itself!**
- So we can pick the other 24 bits of e_{13} however we want, and then **flip those 8 secret pins (bit 18 and bits 25..31) in a single step (`[E21]`)** so that $W_{13} \bmod 256 = \texttt{0x80}$ is guaranteed **100% of the time ($256\times$ speedup)**!

3.2 Explain Like I'm an Undergraduate (ELIU): Derivation of `[E15]` – `[E22]`

1. **Absorbing $W_{15} = 440$ and $W_{14} = 0$ (`[E15]` – `[E17]`):** Given (a_{10}, a_{11}, a_{12}) and the terminal boundary $a_{13} \dots a_{20}$, `[E7]` immediately determines e_{16}, e_{15}, e_{14} (`[E15]`). Setting $W_{15} = 440$ and $W_{14} = 0$ in `[E7]` at $t = 15, 14$ yields `[E16]` :

$$e_{12} = (e_{16} - a_{12} - \Sigma_1(e_{15}) - K_{15} - 440) - \mathrm{Ch}(e_{15}, e_{14}, e_{13}),$$
$$e_{11} = (e_{15} - a_{11} - \Sigma_1(e_{14}) - K_{14}) - \mathrm{Ch}(e_{14}, e_{13}, e_{12}),$$

and `[E7]` at $s = 12, 11$ recovers a_8, a_7 (`[E17]`).

2. **Exact $O(1)$ Pivot Inversion of $W_{13} \bmod 256 = \texttt{0x80}$ (`[E18]` – `[E21]`):** Reducing W_{13} (`[E18]`) modulo 256 gives `[E19]` :

$$\Sigma_1(e_{13}) \equiv R_{\text{lo}} := \big(c_{W_{13}} - \mathrm{Ch}(e_{13,\text{lo}}, e_{12,\text{lo}}, e_{11,\text{lo}}) - \texttt{0x80}\big) \pmod{256}.$$

Notice that R_{lo} depends **only on the low 8 bits** $e_{13,\text{lo}} = e_{13} \bmod 256$! Meanwhile, for each bit $b \in \{0, \dots, 7\}$, the b -th bit of $\Sigma_1(e_{13})$ is (`[E20]`):

$$\big(\Sigma_1(e_{13})\big)_b = (e_{13})_{b+6} \oplus (e_{13})_{b+11} \oplus (e_{13})_{(b+25) \bmod 32}.$$

- For $b \in \{0, \dots, 6\}$, bit $(e_{13})_{b+25} \in \{25, \dots, 31\}$ appears uniquely in row b and nowhere in bits $0..17$.
- For $b = 7$, bit $(e_{13})_{7+11} = (e_{13})_{18}$ appears uniquely in row 7 and nowhere in bits $0..17$. Hence, for **any** assignment to bits $0..17$ of e_{13} ($e_{13,\text{base}}$) and any assignment to bits $19..24$, setting bit 18 and bits $25..31$ via `[E21]` :

$$\Delta := \big(R_{\text{lo}} \oplus \Sigma_1(e_{13,\text{base}})\big) \bmod 256, \qquad e_{13,\text{fixed}} := e_{13,\text{base}} \mid \big((\Delta \wedge \texttt{0x7f}) \ll 25\big) \mid \big(((\Delta \gg 7) \wedge 1) \ll 18\big),$$

satisfies $(W_{13} \bmod 256) = \texttt{0x80}$ with **probability** 1!

3. **Ch-Annihilation & Twin Majority (`[E22]`):** By choosing a_{12} so that $e_{15} = \texttt{0xffffffff}$ and setting $a_{10} = a_{11}$ (`[E22]`), we get $\mathrm{Ch}(e_{15}, e_{14}, e_{13}) = e_{14}$ and $\mathrm{Maj}(a_{11}, a_{10}, a_9) = a_{11}$. Thus e_{12} **and a_8 become 100% constant across all 2^{24} choices of e_{13}!**

Section 4: `Carry-Sudoku` — 7-Stage Causal Bit-Slice & Rotation-Invariant Right-Context Solver (`[E23]` – `[E26]`)

4.1 Explain Like I'm 5 (ELI5): Solving Addition from Right to Left Without Knowing a_6

To build a valid Right Context, we need to find values of e_{13} where a_5 and a_6 are identical twins ($a_5 = a_6$, `[E11]`). Look at the formula for a_5 (`[E9]`):

$$a_5 = X + \mathrm{Maj}(a_8, a_7, a_6), \qquad \text{where } X = \texttt{0xffffffff} - a_9 + \Sigma_0(a_8).$$

Normally, to check if $a_5 = a_6$, you would have to compute all 32 bits of a_9, rotate a_9 through $\Sigma_0(a_9)$ to get a_6, and then add $X + \mathrm{Maj}(a_8, a_7, a_6)$ to get a_5. That means you can't check *anything* until all 32 bits of e_{13} are chosen!

The `Carry-Sudoku` Breakthrough (Theorem 1, `[E23]`): What if we **already assume** a_6 is going to equal a_5, and plug X in place of a_6 inside the Majority gate? Call that test sum $S = X + \mathrm{Ch}(D, X, a_8)$, where $D = a_8 \oplus a_7$ (`[E23]`). Notice two mind-blowing facts about S:

1. **S does NOT use a_6 or $\Sigma_0(a_9)$ at all!**
2. **Every bit where a_8 and a_7 differ ($D_b = 1$), S_b MUST equal X_b (`((S ^ X) & D) == 0`, `[E24]`)!** Why? Because on any bit where $a_8 \neq a_7$, $\mathrm{Maj}(a_8, a_7, a_6)$ equals a_6, so $a_5 = X + a_6 + \mathrm{carry}$ can only equal a_6 if $X + \mathrm{carry} = 0 \pmod 2$, which means S and X must match on bit b!
3. Since addition carries only flow from **right to left (bit 0 $\rightarrow$ bit 31)**, the lowest 4 bits of S, X, and D depend **only on the lowest 4 bits of e_{13}**!

So just like Sudoku, we fill in e_{13} **4 bits at a time from right to left (`[E25]`)**, throwing away any 4-bit chunk where `((S ^ X) & D)` is non-zero! By the time we reach bit 18, **99.999% of wrong e_{13} values have already been eliminated**, before we ever compute $\Sigma_0(a_9)$!

4.2 Explain Like I'm an Undergraduate (ELIU): Proof of Theorem 1 & The 7-Stage Pipeline (`[E23]`–`[E26]`)

Let us review **Theorem 1 (`[E23]`)**: Define $X := \texttt{0xffffffff} - a_9 + \Sigma_0(a_8)$, $D := a_8 \oplus a_7$, and $S := X + \mathrm{Ch}(D, X, a_8) \pmod{2^{32}}$. Then:

$$a_5 = a_6 \iff \Big(((S \oplus X) \wedge D) = 0\Big) \ \wedge \ \Big(((a_6 \oplus S) \wedge \neg D) = 0\Big).$$

- **Why does `((S ^ X) & D) == 0` prune causally (`[E24]`–`[E25]`)?** Because a_8 is constant (`[E22]`), $a_9 = e_{13} - c_{e_{13}}$, and $a_7 = c_{a_7,\text{full}} - (e_{14} \wedge e_{13}) + ((a_{10} \oplus a_8) \wedge a_9)$, every operation in X, a_7, D, and $S = X + \mathrm{Ch}(D, X, a_8)$ is **strictly causal (lower-triangular)**: bits $0..k$ of S, X, D depend exclusively on bits $0..k$ of e_{13}! Thus `solve20_padded_fast.c` and `solve20_padded_cuda.cu` assign bits $0..17$ of e_{13} across 5 causal stages (`[E25]`):
- **Stage 1a (bits 0..3):** checks mask `0xf`.
- **Stage 1b (bits 4..7):** checks mask `0xff`, fixing R_{lo} and $(e_{13})_{18} \oplus (e_{13})_{13}$.
- **Stage 1c (bits 8..11):** checks mask `0xfff`.
- **Stage 1d (bits 12..14 + exact bit 18):** once bit 13 is known, bit 18 is known! Checks mask `0x7fff`.
- **Stage 1e (bits 15..17, checked through bit 18):** checks 19-bit mask `0x7ffff` and pins bits $25..31$ via `[E21]`.
- **Why are bits `13..15` and `3..4` of $\Sigma_0(a_9)$ rotation-invariant (`[E26]`)?** Once bits $0..18$ and $25..31$ of e_{13} are fixed, only the 6 bits $19..24$ remain free. Under $\Sigma_0(a_9) = \mathrm{ROR}(a_9, 2) \oplus \mathrm{ROR}(a_9, 13) \oplus \mathrm{ROR}(a_9, 22)$, bits $19..24$ of a_9 (plus 1 carry bit at position 25) land in $[17..23]$, $[6..12]$, and $[29..31, 0..2]$—completely missing bits **13..15** and **3..4**! Therefore, Stage 2 checks `a6 == a5` on bits `13..15` and `3..4` (`[E26]`) **before** looping over the 6 free bits $19..24$, reaching **86.4 billion trials/s** on 12-core Apple M3 and **56 trillion trials/s (`29,310.3 ctx/s`)** on 1× NVIDIA L4 GPU!

Section 5: `Slot-Sudoku` & Zero-Table 5-Instruction $\mathbb{F}_2$ Inverse (`[E27]` – `[E37]`)

5.1 Explain Like I'm 5 (ELI5): The 5-Step Origami Unfolder and the 4-Slot Pigeonholes

Now that Team Right can generate thousands of exact $D = 0$ padded contexts per second, Team Left sweeps $a_1 \in [0, 2^{32})$ and solves the three chained equations (`[E14]`):

- **Level 1:** Find W_1 such that $Q_0(W_1) = T_{16} + F_1(a_1)$.
- **Level 2:** Find W_2 such that $Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1$.
- **Level 3:** Find W_3 such that $Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2$.

Here we hit two huge engineering challenges—and solve both with **origami math (`Theorem 2`)** and `Slot-Sudoku` :

1. **Cut Memory in Half with the 5-Step Origami Unfolder (`[E27]` – `[E29]`):** Storing a 32-bit number u in every table slot takes 4 bytes per slot. What if we only store the **bottom 16 bits (u_{lo})** in each slot (cutting table size in half to 2 bytes per slot!)? Previously, recovering the top 16 bits (u_{hi}) required looking up a 128 KB table in memory 200 billion times per sweep, clogging the GPU's cache. In **Theorem 2 (`[E28]` – `[E29]`)**, we prove that you don't need any lookup table at all: because $\sigma_0(u_{\text{hi}} \ll 16)$ shifts bits in a neat triangle, **5 simple bit-shift/XOR instructions (`reconstruct_u`) unfold the exact top 16 bits directly inside CPU/GPU registers**!
2. **Pack the Pigeonholes with `Slot-Sudoku` (`[E30]` – `[E33]`):** Some target values y have 0 solutions to $Q_0(u) = y$, while others have 1, 2, 3, or 4 solutions.

- In our **6.00 GB `Slot-Sudoku` table (`[E30]` – `[E31]`)**, every group of 16 adjacent y's shares 12 slots arranged by Earliest-Deadline-First (EDF) sliding windows plus a Cuckoo-Sudoku 4th overflow slot, capturing **82.98%** of all preimages in just 6 GB!
- In our **16.00 GB Direct-Indexed 2-Way Exact GPU table (`[E32]`)**, every 32-bit y directly indexes its own `uint32_t` holding up to two exact 16-bit roots $(u_{\text{lo},1}, u_{\text{lo},2})$—giving **zero false-positive checks** across all 3 levels!
- And every survivor of Level 3 is a **complete 19-step padded preimage**, which we test in 1 step (`[E36]` – `[E37]`) to find our **20-step padded preimage**!

5.2 Explain Like I'm an Undergraduate (ELIU): Theorem 2 (`reconstruct_u`), SWAR Filtering, & σ_0^{-1} (`[E27]` – `[E37]`)

1. **Proof of Theorem 2 — 5-Instruction Closed-Form Inverse (`[E27]` – `[E29]`):** Given $y = Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}$ and the low 16 bits $u_{\text{lo}} = u \bmod 2^{16}$, let $U = u_{\text{hi}} \ll 16$. Since $(u + y) \bmod 2^{17} = \sigma_0(u) \bmod 2^{17}$ and $\sigma_0(u) = \sigma_0(U) \oplus \sigma_0(u_{\text{lo}})$, we know bits $0..16$ of $\sigma_0(U)$ (`[E27]`):

$$\sigma_0(U) \equiv v := (u_{\text{lo}} + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{17}}.$$

Now write $U = L(T) := T \oplus (T \ll 11) \oplus (T \ll 15) \pmod{2^{32}}$ for an upper-half word T. Why this specific operator $L(T)$? Because inside $\sigma_0(U) = \text{ROR}(U, 7) \oplus \text{ROR}(U, 18) \oplus (U \gg 3)$, the term $\text{ROR}(U, 18) = (U \gg 18) \oplus (U \ll 14)$ interacts with $(U \gg 7)$ and $(U \gg 3)$ at shifts $+11$ and $+15$! Substituting $U = T \oplus (T \ll 11) \oplus (T \ll 15)$ cancels the two upper bands, leaving a direct closed-form expression for T (`[E28]`):

$$b_0 = \big((v \ll 15) \oplus (v \ll 11) \oplus v\big) \wedge \texttt{0x00010000},$$
$$b_{12} = \big((v \ll 14) \oplus (v \ll 3)\big) \wedge \texttt{0x00060000},$$
$$T = b_0 \mid b_{12} \mid \big((v \ll 18) \wedge \texttt{0xfff80000}\big),$$

followed by $u = u_{\text{lo}} \mid \big(T \oplus (T \ll 11) \oplus (T \ll 15)\big)$ (`[E29]`).

2. **64-Bit SWAR Pre-Reconstruction Filter (`[E34]` – `[E35]`):** At Level 3, before reconstructing u_{hi}, notice that the majority condition $\big((a_4 \oplus a_5) \wedge M_3\big) = 0$ (`[E13]`) on bits $0..14$ depends only on u_{lo}! Using SIMD-Within-A-Register (SWAR, `[E35]`), we test 4 candidate 16-bit slots packed in a 64-bit word `w` in 3 instructions, rejecting $> 99\%$ of quads without unpacking a single slot!

3. **Terminal 20-Step Bijection (`[E36]` – `[E37]`):** Every candidate passing Level 3 satisfies steps $0..18$ and all 72 padding bits—so it is a **complete 19-step padded preimage**! To extend to **20 steps**, we only need step 19's schedule equation (`[E36]`):

$$\sigma_0(W_4) + W_3 \equiv T_{19} := W_{19} - \sigma_1(W_{17}) - W_{12} \pmod{2^{32}}.$$

Since σ_0 is a bijection over $\mathbb{F}_2^{32}$ ($\det_{\mathbb{F}_2}(\sigma_0) = 1$, `[E37]`), $W_4 = \sigma_0^{-1}\big((T_{19} - W_3) \bmod 2^{32}\big)$ is uniquely determined by W_3 and T_{19}, yielding a 2^{-32} filter per 19-step padded solution!

Section 6: GPU Warp-Sudoku & The Zero-a_3 Unpadded Bridge (`[E38]` – `[E41]`)

6.1 Explain Like I'm 5 (ELI5): Keeping All 32 GPU Lanes Busy + The Unpadded Shortcut

1. **Why GPUs Need `Warp-Sudoku` (`§6.4` of Main Paper):** On an NVIDIA L4 GPU, threads work in locked teams of 32 (**Warps**). At Level 2, only a few threads in a warp find a valid a_3 with small Hamming weight (`[E34]`). If those few threads immediately run Level 3 while the other threads wait, most of the GPU sits idle!

- **Shared-Memory Level-3 Inbox:** Instead, whenever a thread finds a Level-2 survivor, it drops the task into a fast shared-memory inbox (`__ballot_sync`) and keeps working on Level 2. Whenever the inbox has 32 tasks ready, all 32 threads grab one task each and run Level 3 at **100% parallel efficiency**!
- **16-Way Single-T_{16} Coupon ($B = 16$):** Notice that Level 1 (`Q0(W1) = T16 + F1(a1)`) only depends on the Right context through the single number T_{16}! By grouping $B = 16$ Right contexts that share the same T_{16}, the GPU does the Level-1 lookup **once** and reuses it across 16 contexts—reaching **180,831.9 exact 19-step padded solutions/sec** on a single L4 GPU!

2. **The Unpadded 20-Round Shortcut (`Zero-a3 Bridge` , `[E38]` – `[E41]`):** What if the postal inspector doesn't require padding (the unpadded 20-round compression function)? Then we can set cell $a_3 = 0$ along with $a_5 = a_6 = a_8 = a_9 = 0$ (`[E38]`). Since two zeros always win a 2-out-of-3 vote (`Maj(0, a4, 0) = 0` , `[E39]`), voter a_4 is defeated **100% of the time without any Hamming weight filter loss**, solving 20-step unpadded SHA-256 in **21.1 minutes on 16 CPU cores** (`[E41]`)!

6.2 Explain Like I'm an Undergraduate (ELIU): Summary of Benchmarks & Quick Verification

Solver / Architecture	Table Structure	Right-Context Speed ($D = 0$)	19-Step Padded Rate (`sol19/s`)	Expected 20-Step Padded Time (1 Device)	Expected Time on 8× L4 GPUs
Carry/Slot-Sudoku CPU (`solve20_padded_-fast.c` , 12 M3 threads)	6.0 GB 2-Tier Nested 4-Slot Sudoku	`86.4B trials/s` (`0.11s` /ctx)	`6,242.8 sol19/s`	`11.9 hours`	—
Cycle-0 CUDA Baseline (1× L4 GPU)	6.0 GB 2-Tier Sudoku ($B = 2$)	`8,370 ctx/s`	`53,847.5 sol19/s`	`1.38 hours`	`10.4 min`
Upgraded 6 GB CUDA (`solve20_padded_cud-a.cu` , 1× L4 GPU)	6.0 GB + Zero-Table `[E29]` + $B = 8$	`7,580 ctx/s`	`73,560.5 sol19/s`	`1.06 hours`	`8.0 min`
Cycle-1 CUDA (`16 GB Direct 2-Way` , 1× L4 GPU)	16.0 GB Direct 2-Way Exact (`[E32]`)	`8,370 ctx/s`	`132,888.7 sol19/s` (`2.47×`)	`33.7 min`	`4.2 min`
Cycle-2 CUDA (`16 GB Direct + Multi-T16 + Warp-L3` , 1× L4)	16.0 GB Direct + 16-Way T_{16} + Warp L3	**`29,310.3 ctx/s`** (`3.50×`)	**`180,831.9 sol19/s`** (`3.36×`)	**`24.7 min`**	**`3.1 min`**

Standalone Verification (`verify_all_witnesses.py`)

Run the zero-dependency Python verifier to check all three witnesses (20-step padded, 19-step padded, and 20-step unpadded) and all internal Carry-Sudoku invariants (`a5 == a6 == 0x67978bcd` , `a10 == a11 == 0xa7b351d7` , `e9 == e10 == e15 == 0xffffffff` , `W13 & 0xff == 0x80` , `W14 == 0` , `W15 == 440`):

```
python3 verify_all_witnesses.py
```